

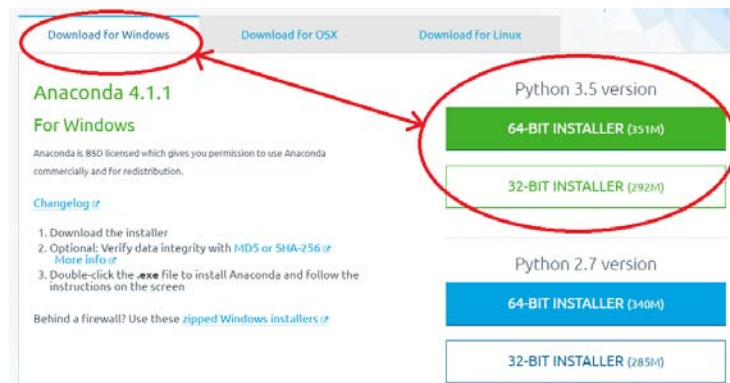
Getting Started With Python

This handout will cover how to set up Python and introduce you to Spyder, the Python development environment we will be using throughout this course. It will also show you how to check that the Python library packages matplotlib and numpy were successfully installed. They will be used later in this class to work with charts.

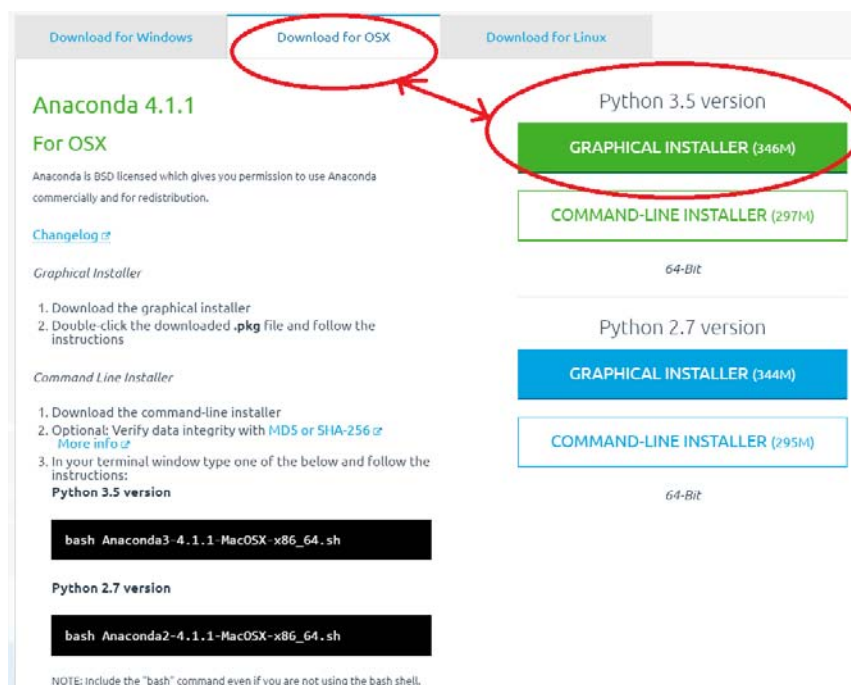
Downloading the Python Installer

To download the free “Anaconda” Python distribution created by Contium Analytics:

1. Visit the [download page](#) for Contium Analytics. NOTE: the installer we'll be downloading and installing is larger than the average file, because it contains Python, associated packages, a code editor, and some other toys. It may take 15-20 minutes in total to download and install when executing the commands.
2. Windows Operating System: There are two versions of the installer, one for 32-bit Windows, and one for 64-bit Windows. Microsoft provides instructions [here](#) to determine (relatively easily) if you have a 32 or 64-bit setup. If you are running 32-bit Windows, be sure that you select the 32-bit version, and if you are running 64-bit Windows, you can download either version.
The class is taught using Python 3.5. Do not download any other version.

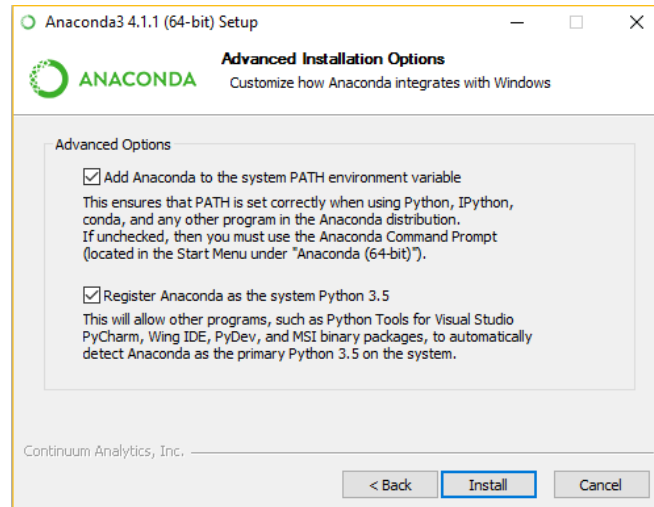


3. Macintosh Operating System: There is only one version of the installer: the 64-bit version. Contium does not offer a 32-bit installer for Macintosh, since Macs post-2011 are [64-bit only](#) (and post-2007 are 64-bit compatible).
The class is taught using Python 3.5. Do not download any other version.



Installing Python and Spyder

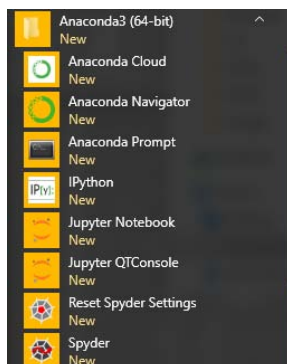
1. To install Python after downloading the graphical installer, double click the .exe (Windows) or .pkg (Mac) file and follow the instructions on the screen. There are some additional notes and comments on installing/uninstalling Python on Contium's website [here](#).
2. Note that Contium's Python distribution should (per their [quick start guide](#)) install cleanly into a single directory, should not require Administrator or root privileges, and should not affect other Python installs on your system (if you have any), or interfere with OSX Frameworks. Let us know via Piazza if you're having any issues, so that others can learn along with you.
3. When installing Anaconda, a pop-up menu may ask whether or not to "Add Anaconda to my PATH environment variable", and "Register Anaconda as my default Python 3.5." We suggest accepting both options for the purposes of this class.



Starting Spyder, a code editor

When you download and install the Anaconda distribution of Python, you are getting several tools related to Python development. One of the tools is Spyder (Scientific Python Development Environment), an [integrated development environment](#) useful for writing, running, and debugging code. There are two primary ways to access the tools and open Spyder:

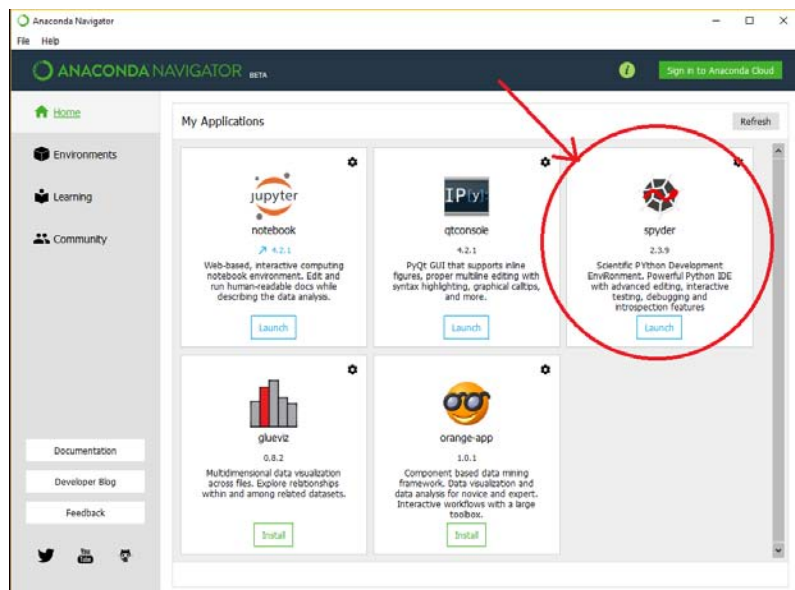
1. The Anaconda folder from the Start menu [Windows Only].
2. The Anaconda Navigator shortcut icon. It should open the Anaconda Navigator. Click the blue "Launch" button by "spyder" to launch Spyder. It should take a moment to load and may appear to pause for a few minutes.



(1)



(2)

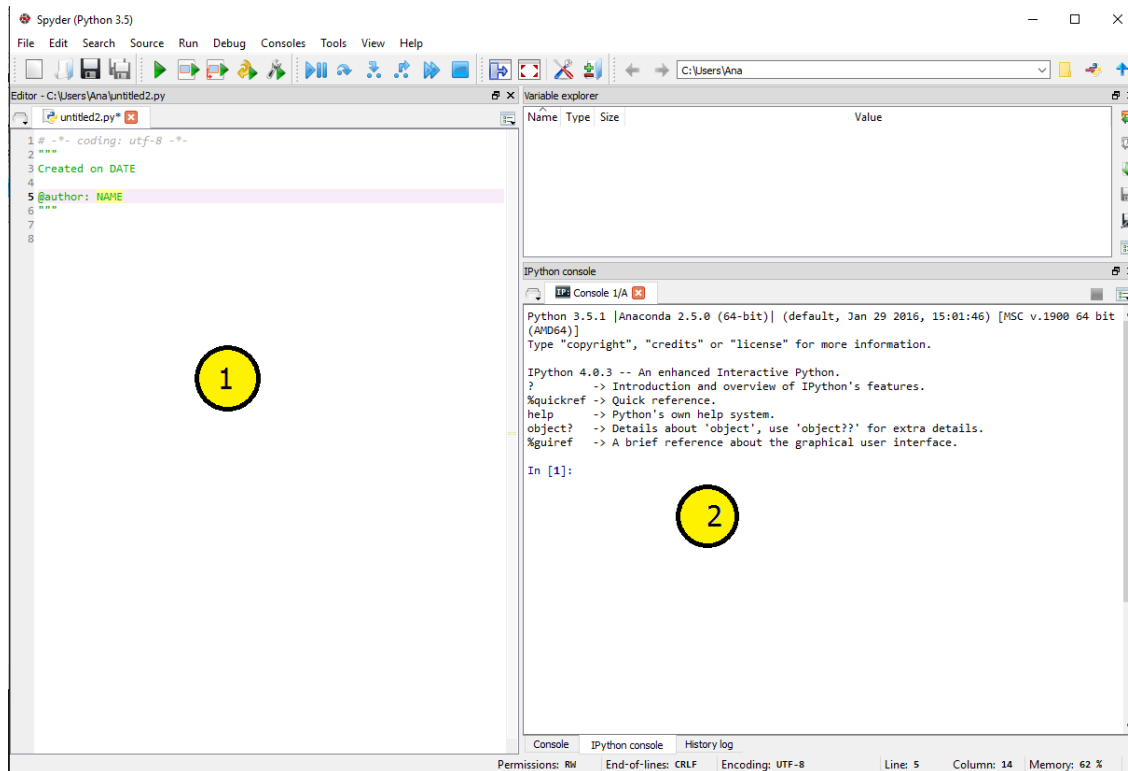


Spyder Application Overview

Opening the Spyder application should present you with the following window (screenshot is from Windows, but similar on Macs). This window contains two commonly used parts, among others:

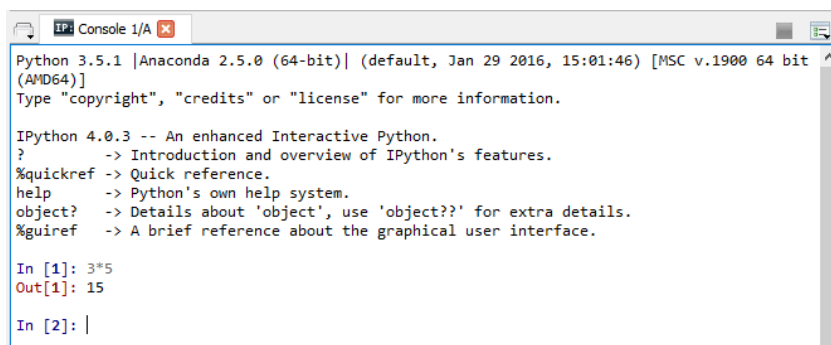
- (1) Code Editor that lets you create and edit existing Python source files [YELLOW CIRCLE 1]
- (2) IPython interpreter pane, which gives you access to the Python interactive mode [YELLOW CIRCLE 2].

During the following discussion of Spyder's features, you should start Spyder and try to replicate the screenshots.



Using the IPython prompt:

1. The IPython prompt looks something like this: "In [1]:", and can be used to run Python code. You can type Python code directly into this prompt, and pressing enter executes the code fragment.
2. Try typing the following after the prompt and pressing the enter key: `3*5`



Using the IPython prompt – examples and exercises:

Addition (+), subtraction (-), multiplication (*), division (/), modulo (%) and power (**) operators are built into the Python language. This means you can use them right away. If you want to use a square root in your calculation, you can either raise something to the power of 0.5 or you can *import* the *math* module. Do not worry about what it means right now, we will cover this later during the course. Below are two examples of square root calculation:

```
In [1]: 16**0.5
Out [1]: 4.0
```

```
In [2]: import math
```

```
In [3]: math.sqrt(16)
Out[3]: 4.0
```

The math module allows you to do a number of useful operations:

```
In [1]: math.log(16, 2)
Out [1]: 4.0
```

```
In [2]: math.cos( 0 )
Out [2]: 1.0
```

Exercises: Use the IPython prompt to calculate:

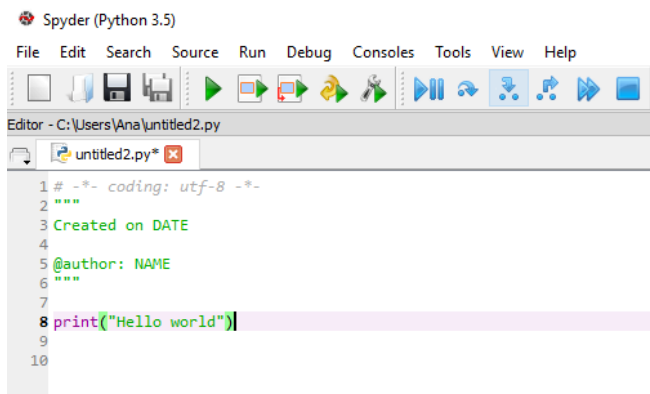
(these are just for practice, solutions will not be graded or collected in class)

1. $6+4*10$
2. $(6+4)*10$
 - a. (Compare this to #1, and note that Python uses parentheses just like you would in normal math to determine order of operations!)
3. 23.0 to the 5th power
4. Positive root of the following equation:
 - a. $34*x^2 + 68*x - 510$
 - b. Recall:
 - i. Given $a*x^2 + b*x + c$,
 - ii. Then $x1 = (-b + \sqrt{b*b - 4*a*c})/(2*a)$
5. `import math`
`math.cos(3.4)**2+math.sin(3.4)**2`

To create, save, and run a file:

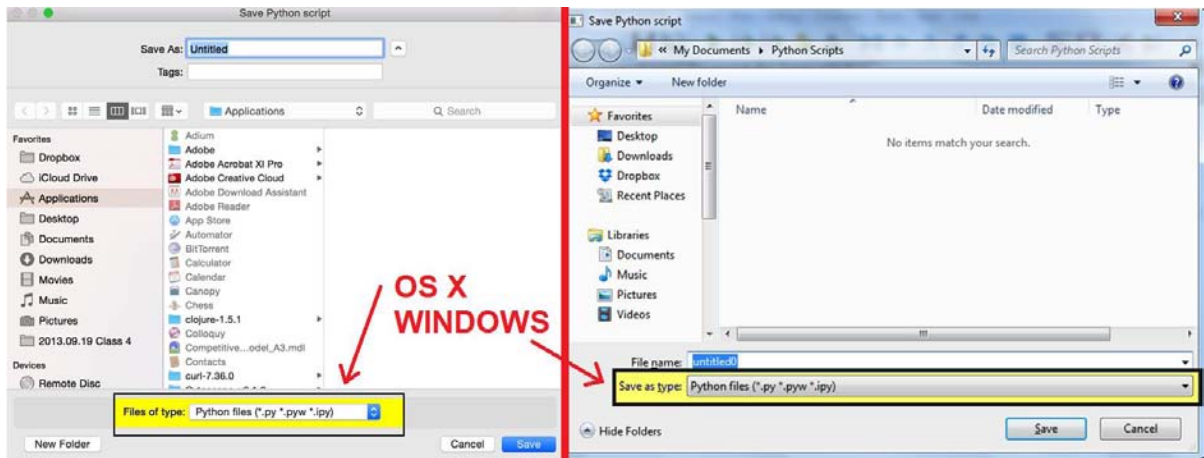
Creating the file

1. In Spyder's File menu, select "New file".
2. In the new file, type the following: `print 'hello world'`



Saving the file

1. Saving your files in a location specific for this course can aid in organization. To do that, first create a directory (or folder) for your 6.0001 material, with an appropriate name.
2. Now you can save your "hello world" file in the new folder that you just created.
3. From the Spyder File menu, click "Save As" and then navigate to your course folder before typing a name for this file, e.g., "pyTest". Make sure that you're saving the file as type Python (see below for images on Mac and PC).
4. Click the Save button



Running the file

1. From within Spyder, go to the "Run" menu, click on "Run".
2. You should see two parts to the output in the IPython console. First, The runfile(...) line shows you the filepath on your computer for the file you just ran. Second, the output of the file, "hello, world!", should appear.

```
Python 2.7.9 [Anaconda 2.1.0 (x86_64)] (default, Dec 15 2014, 10:37:34)
Type "copyright", "credits" or "license" for more information.

IPython 2.3.1 -- An enhanced Interactive Python.
Anaconda is brought to you by Continuum Analytics.
Please check out: http://continuum.io/thanks and https://binstar.org
?      -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help    -> Python's own help system.
object? -> Details about 'object', use 'object??' for extra details.
%gui?   -> A brief reference about the graphical user interface.

In [1]: runfile('/Applications/Untitled.py', wdir='/Applications')
hello, world!

In [2]:
```

3. Go back to the Code Editor and add the following line: `print 'I like 6.001!'`
4. Select "Run File" again (you can also use the green triangular Run button on the toolbar or the shortcut F5), and observe the change in result.
5. Close your test file by clicking the X in its filename tab.
6. Now reopen your test file by double-clicking its name in the File menu's "Open recent" option.
7. Congratulations - your Python environment has been installed, and you now know how to enter expressions in Python and how to save, run, and open files!
8. For additional help on Spyder, go the Help menu within Spyder, and select the "Spyder tutorial" option.

A Very Simple Program: Raising a number to a power and taking a logarithm

The goal of this programming exercise is to make sure your python and numpy installations are correct, to get you more comfortable with using Spyder, and to begin using simple elements of Python. Standard elements of a program include the ability to print out results (using the `print` operation), the ability to read input from a user at the console (for example using the `input` function), and the ability to store values in a variable, so that the program can access that value as needed.

Assignment:

Write a program that does the following in order:

1. Asks the user to enter a number "x"
2. Asks the user to enter a number "y"
3. Prints out number "x", raised to the power "y".
4. Prints out the log (base 2) of "x".

Use Spyder to create your program, and save your code in a file named 'ps0.py'. An example of an interaction with your program is shown below. The words printed in blue are ones the computer should print, based on your commands, while the words in black are an example of a user's input. The colors are simply here to help you distinguish the two components.

```
Enter number x: 2
Enter number y: 3
x**y = 8
log(x) = 1
```

Hints:

- To see how to use the `print` command, you may find it convenient to look at the [input](#) and [output](#) of the Python Wikibook. This will show you how to use print statements to print out values of variables.
- To see how to read input from a user's console into the Python environment, you may find it convenient to look at the same section (see for example the `input()` function)
- Reference the [basic math section](#) of the Python Wikibook to read more about using basic mathematical operators in Python

- To take the logarithm of a variable, import either of the numpy or pylab packages. You can then call either `numpy.log2` or `pylab.log2` to calculate the logarithm. See the Getting Started document on importing packages and the many Numpy [examples](#) online for more info. Googling the log2 function may take you [here](#), which has some helpful info.
- Remember that if you want to hold onto a value, you need to store it in a *variable* (i.e., give it a name to which you can refer when you want that value). You may find it convenient to look at the [variables and strings](#) section of the Python Wikibook. (As you read through, remember that in Python 3.x you should be using `input()` not `raw_input()`). Take a look at the “[Combining Numbers and Strings](#)” sub-section, because you will be working with numbers and strings in this problem and will have to convert between the two using the `str()` and `int()` functions.